



MOBILEWRIGHT · ANDROID AUTOMATION

MobileWright Tutorial & Reference

From Zero to Your First Green Test — a guided, step-by-step path to writing and running native Android tests, plus a complete API reference.

A Playwright-style framework for native Android & iOS automation, taught through this repository's working test suite against the Mobile QA Sandbox app.

by [Haris Chandra Roy](#)

Personal, unofficial project · provided "as is" for educational use · see the colophon on the final page for full attribution and legal notice.

MobileWright Tutorial — From Zero to Your First Green Test

A step-by-step, hands-on tutorial for the **MobileWright Android test suite**. It walks you from an empty machine to writing, running, and debugging your own mobile test against the [Mobile QA Sandbox](#) app (`com.haris.testapp`).

Audience: QA engineers and developers new to mobile automation. **Official MobileWright documentation:** <https://mobilewright.dev/> A printable A4 version is available at [MOBILEWRIGHT_TUTORIAL.pdf](#) . This single guide covers everything — from first setup to a complete API reference (see [Part 6 — Reference](#)). Author, attribution, and the full legal notice are on the final page (see [About This Document](#)).

Table of Contents

Part 1 — Understand

1. [What You Will Build](#)
2. [How MobileWright Works](#)
3. [Tour of This Repository](#)

Part 2 — Set Up 4. [Install the Toolchain](#) 5. [Get a Device Ready](#) 6. [Install the App and the Agent](#) 7. [Run the Suite for the First Time](#)

Part 3 — Learn by Reading 8. [Lesson 1: Anatomy of a Test](#) 9. [Lesson 2: Finding Elements \(Locators\)](#) 10. [Lesson 3: The Helper Factory](#) 11. [Lesson 4: The Full Account Lifecycle](#) 12. [Lesson 5: Negative Tests](#) 13. [Lesson 6: Device Features and Gestures](#)

Part 4 — Write Your Own 14. [Workshop: Write a New Test](#) 15. [Discovering Locators with `inspect.spec.ts`](#) 16. [Tagging and Selective Runs](#)

Part 5 — Operate 17. [Reading the Report](#) 18. [When Things Go Wrong](#) 19. [Checklist Before You Push](#) 20. [Where to Go Next](#)

Part 6 — Reference (*the cheat sheet, folded in*) 21. [CLI Command Reference](#) 22. [ADB Command Reference](#) 23. [Emulator Management](#) 24. [Locator Strategies \(full\)](#) 25. [Gestures \(full\)](#) 26. [Assertions & Wait Strategies](#) 27. [Reading the View Tree](#) 28. [Runtime Permissions](#) 29. [API Testing](#) 30. [Framework Design Patterns](#) 31. [Best Practices](#) 32. [Interview Questions](#) 33. [Quick-Reference Tables](#)

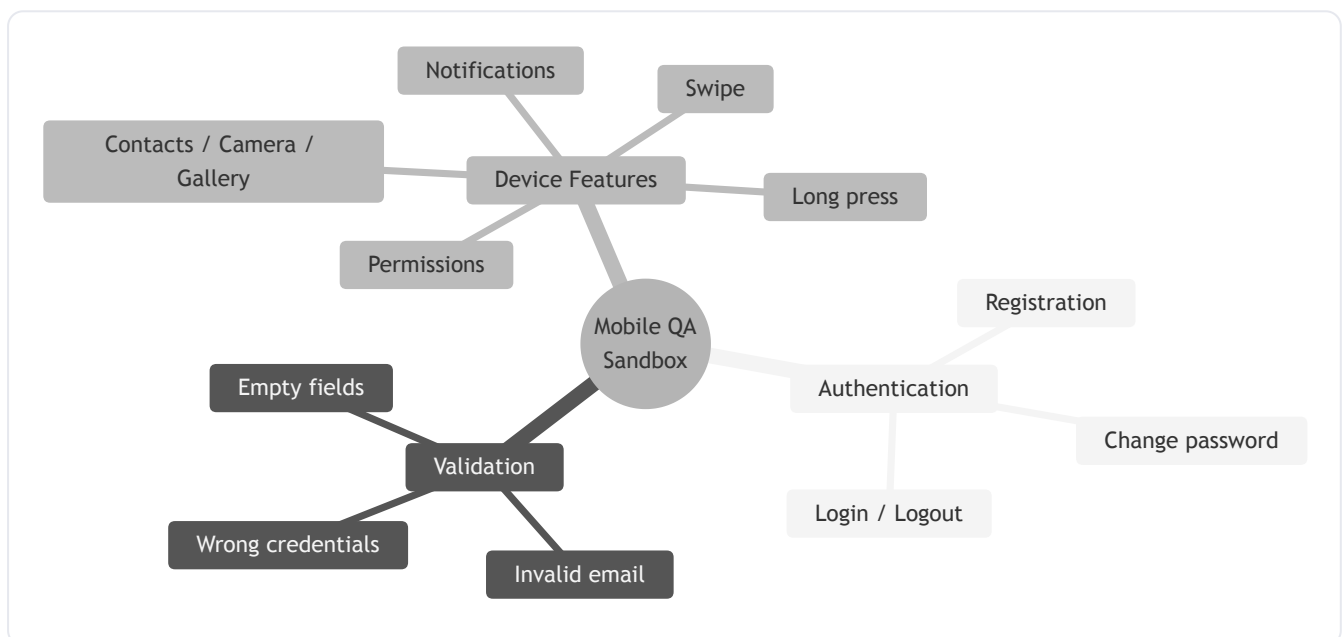
Part 1 — Understand

1. What You Will Build

By the end of this tutorial you will have:

- A working MobileWright environment (Node, JDK, Android SDK, a device/emulator).
- The **Mobile QA Sandbox** app and the MobileWright on-device agent installed.
- A green run of the existing suite — 35+ tests covering registration, login, profile updates, password changes, validation paths, and device features.
- **Your own new test**, written from scratch, tagged, and passing.

This suite validates a personal-information Android app end to end:

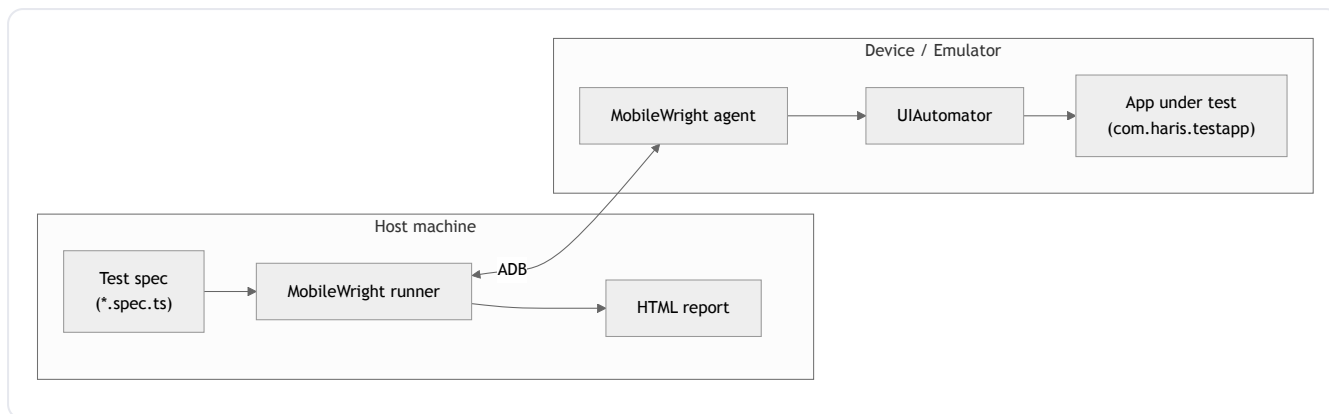


Tip: You do not need to write any app code. The app under test already exists; you are writing the tests that drive it.

2. How MobileWright Works

MobileWright brings the Playwright developer experience to **native mobile**. You write TypeScript specs using `test`, `expect`, and the `screen / device` fixtures. The runner drives a small **on-device agent** over ADB, which performs UIAutomator actions on the device and returns the accessibility tree.

Reference: this section is a practical overview. For the authoritative API and concepts, see the official MobileWright documentation at <https://mobilewright.dev/>.



If you know Playwright for the web, the mental model transfers almost directly:

Playwright (web)	MobileWright (mobile)
page	screen
browser / context	device
page.goto(url)	device.launchApp(bundleId)
page.getByRole(...)	screen.getByLabel / getByTestId / getByRole / getByText
expect(locator).toBeVisible()	expect(locator).toBeVisible()
CSS / XPath	accessibility label, resource ID, role, text, type

Why not Appium? No external server and no desired-capabilities boilerplate. One CLI command (`npm install mobilewright`) puts the agent on the device, and your specs run directly in Node.

3. Tour of This Repository

```

mobilewright/
├── tests/
│   ├── basic_testApp.spec.ts      # BEGINNER – self-contained smoke + happy path (2
tests)
│   └── advance/
│       ├── testApp.spec.ts        # happy path + 12 negative tests (13 tests)
│       └── explore.spec.ts        # device features: gestures, permissions,
contacts...
│   └── utiles.ts                  # shared helper factory, USER fixture, ERROR
strings
│   ├── inspect.spec.ts            # dump the accessibility tree to discover locators
│   └── mobilewright.config.ts      # config scoped to the advance/ suite
├── mobilewright.config.ts          # root config (testDir: ./tests)
├── package.json                   # scripts + dependencies
├── scripts/generate-docs.mjs       # renders this guide to an A4 PDF
└── MOBILEWRIGHT_TUTORIAL.md       # this guide (tutorial + full reference)
  
```

Read the files in this order to learn fastest:

1. `tests/basic_testApp.spec.ts` — the gentlest example; helpers are inline.
2. `tests/advance/utiles.ts` — the shared helper factory everything else uses.
3. `tests/advance/testApp.spec.ts` — the real-world auth/profile flows.
4. `tests/advance/explore.spec.ts` — gestures, permissions, and system apps.

Part 2 — Set Up

4. Install the Toolchain

Software	Minimum	Why
Node.js	18 LTS	Runs the MobileWright CLI and your TS specs
Java / JDK	17	Required by the Android SDK / <code>adb</code> toolchain
Android Studio	latest	Bundles the SDK, platform-tools (ADB), and emulator
Git	any recent	Clone the repo

```
# 1. Node – verify after installing the LTS from nodejs.org
node --version          # v18.x or higher

# 2. JDK 17 (macOS / Homebrew shown; see README for Linux/Windows)
brew install --cask temurin17
export JAVA_HOME="$(/usr/libexec/java_home -v 17)"
java -version           # 17.x

# 3. Android SDK / ADB on PATH (macOS shown)
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$PATH:$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator"
adb version
```

One-shot check: after cloning and installing, run `npx mobilewright doctor`. It validates Node, JDK, ADB, connected devices, and the agent in a single command — run it first whenever something looks off.

Clone the repo and install dependencies:

```
git clone https://github.com/sharisroy/mobilewright-testapp.git
cd mobilewright-testapp
npm install
```

This installs `mobilewright` (the CLI + runner), `@mobilewright/test` (the test API), and `@types/node`.

5. Get a Device Ready

You need **one** of: an Android emulator or a physical Android phone.

Emulator

```
emulator -list-avds           # see available virtual devices
emulator -avd Pixel_7_API_34 & # boot one (API 31+ recommended)
adb wait-for-device
adb shell getprop sys.boot_completed # prints 1 when ready
```

Create an AVD from **Android Studio** → **Device Manager** → **Create Device** → **Pixel** → **API 31+**.

Physical phone

1. **Settings** → **About phone** → tap **Build number** 7 times to unlock Developer Options.
2. **Settings** → **System** → **Developer Options** → enable **USB debugging**.
3. Connect via USB and accept the *Allow USB debugging?* prompt.

```
adb devices
# emulator-5554    device    ← emulator
# R5CT30XXXXX     device    ← physical phone
```

Both connected? This repo auto-selects the **physical device** so the driver and `raw adb` calls target the same one (see [mobilewright.config.ts](#) and `getAdbSerial` in [tests/advance/utiles.ts](#)). To force a target, set `MW_DEVICE_ID` or `ANDROID_SERIAL` to its serial.

6. Install the App and the Agent

```
# 1. Download the app under test (APK) from its release
gh release download v1.0.0 \
  --repo sharisroy/mobile-qa-sandbox \
  --pattern '*.apk' --dir ~/Downloads

# 2. Install it on the device
adb install ~/Downloads/mobile-qa-sandbox.apk
adb shell pm list packages | grep com.haris.testapp # → package:com.haris.testapp

# 3. Install the MobileWright on-device agent (once per device)
npx mobilewright install
# Expected: { "status": "ok", "data": { "message": "Agent installed successfully" } }
```

The agent must be re-installed after a full device reboot or a wipe. If you ever see `agent not installed`, just run `npx mobilewright install` again.

7. Run the Suite for the First Time

Start small — the beginner smoke test:

```
npx mobilewright test tests/basic_testApp.spec.ts --grep @smoke
npx mobilewright show-report      # open the HTML report
```

Then run progressively more:

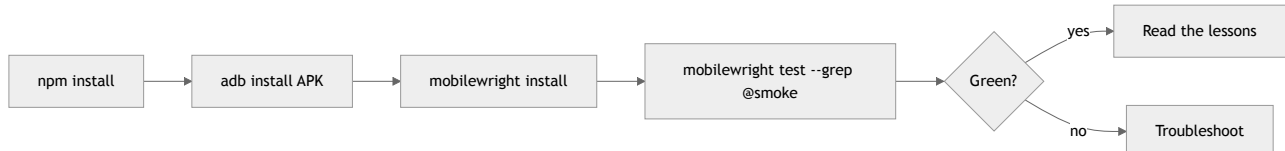
```
# The beginner happy path (register → login → update profile)
npx mobilewright test tests/basic_testApp.spec.ts

# The advanced auth + negative suite
npx mobilewright test tests/advance/testApp.spec.ts

# Device features (gestures, permissions, contacts, camera, notifications)
npx mobilewright test tests/advance/explore.spec.ts

# Everything under ./tests
npm test
```

You should see green. If not, jump to [When Things Go Wrong](#).



Part 3 — Learn by Reading

8. Lesson 1: Anatomy of a Test

Open `tests/basic_testApp.spec.ts`. Every test follows the same shape:

```
import { test, expect } from '@mobilewright/test';

test.setTimeout(600_000); // raise the TEST ceiling for long
flows
test.use({ bundleId: 'com.haris.testapp' }); // app under test

test('basic user flow: register, login, and update profile',
  { tag: ['@regression'] }, // tag for selective runs
  async ({ screen, device }) => { // fixtures injected by the runner

    await device.launchApp('com.haris.testapp'); // 1. arrange
    await expect(screen.getByLabel('App Name')).toBeVisible(); // 2. act /
    assert

    const appName = await screen.getByLabel('App Name').getText();
    expect(appName).toBe('Test Application');
  });
```

Three things to notice:

- **screen** is your handle to the UI — locators and reads come from it.
- **device** controls the app lifecycle and low-level gestures (`device.driver`).
- **timeout VS test.setTimeout()** — the config `timeout` (30 s) is the *per-action* ceiling for one locator or RPC. Long end-to-end tests raise the **test** ceiling separately with `test.setTimeout(600_000)` . The two are independent.

9. Lesson 2: Finding Elements (Locators)

MobileWright offers several ways to locate an element. **Always prefer the most stable one.** This repo's locator priority:

```
getByLabel → getByTestId → getByRole → getByText → getByType
```


Method	Use when	Example from this repo
<code>getByLabel(l)</code>	Element has an accessibility label (best)	<code>screen.getByLabel('Sign In Title')</code>
<code>getById(id)</code>	Element has a resource ID	<code>screen.getById('com.haris.testapp:id/btn_login')</code>
<code>getByRole(r, {name})</code>	Cross-platform semantic role	<code>screen.getByRole('button', { name: 'Login' })</code>
<code>getByText(t)</code>	Text is unique on screen	<code>screen.getByText('My Profile')</code>
<code>getByType(t)</code>	Last resort, by widget type	<code>screen.getByType('EditText')</code>

Because resource IDs are long and repetitive, the helpers build a tiny DRY helper:

```
const id = (value: string) => `${APP_ID}:id/${value}`;
// id('btn_login') → 'com.haris.testapp:id/btn_login'
await screen.getById(id('btn_login')).tap();
```

Rule of thumb: never hard-code `com.haris.testapp:id/...` in a new test — use the `id()` helper. And never reach for `getByText` on a button you could target by label or ID.

10. Lesson 3: The Helper Factory

Open `tests/advance/utiles.ts`. Instead of page objects, this repo uses a **helper factory**: `createHelpers(screen, device, APP_ID)` returns a closure of helpers scoped to one test fixture.

```
import { createHelpers } from './utiles';

test('full lifecycle', async ({ screen, device }) => {
  const { freshLaunchToLogin, registerAndGoToLogin, loginAndGoToProfile, id }
    = createHelpers(screen, device, APP_ID);

  await freshLaunchToLogin(); // terminate + `pm clear` + relaunch → Sign In
  screen
  await registerAndGoToLogin(); // fill the register form, submit, land on login
  await loginAndGoToProfile(); // fill login, submit, land on My Profile
});
```

Key helpers and what they solve:

Helper	What it does	Why it exists
<code>freshLaunchToLogin()</code>	Clears app data and relaunches	Every test starts from a clean slate — no inter-test dependencies
<code>registerAndGoToLogin()</code>	Registers the <code>USER</code> fixture	Reusable arrange step
<code>loginAndGoToProfile()</code>	Logs in and waits for My Profile	Reusable arrange step
<code>fillFreshText()</code>	Tap → fill with small settles	Reliable typing into native fields
<code>clearAndFillById()</code>	Clears a field then types + verifies	Editing pre-filled fields
<code>dismissKeyboard()</code>	Presses BACK to hide the soft keyboard	Keyboard can overlap the next field's tap target
<code>selectDate()</code>	Drives the Material date picker	The picker is fiddly; this isolates it
<code>selectDropdownItem()</code>	Taps below a field to hit a popup option	<code>AutoCompleteView</code> popups aren't in the <code>activity</code> tree
<code>findVisibleElement([...], maxSwipes)</code>	Scrolls until any candidate locator resolves	Self-healing locators for off-screen elements
<code>waitForErrorVisible(text)</code>	Polls the tree for an error string	Soft assertion — returns a boolean, never throws mid-poll

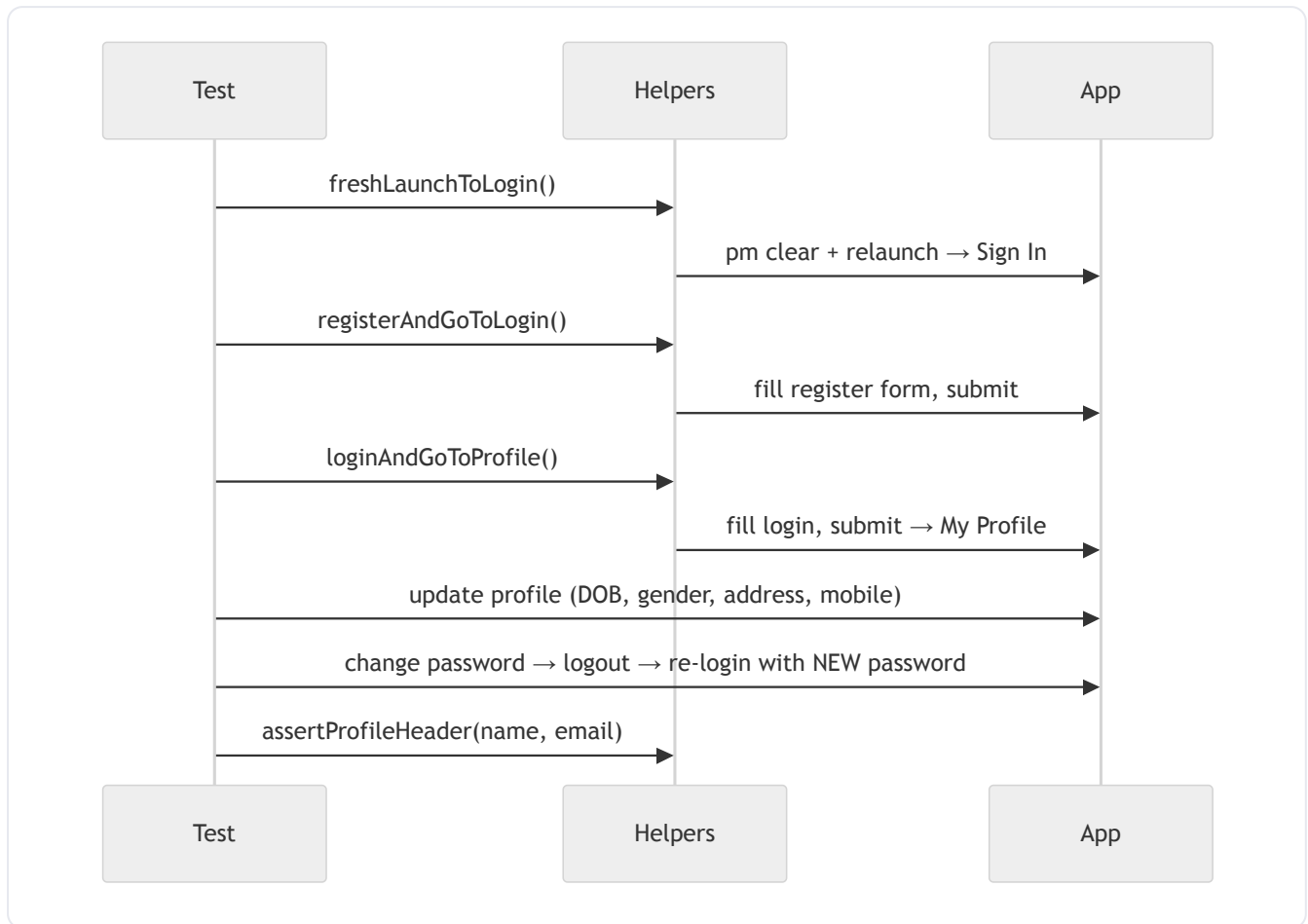
The fixture data is deterministic and unique per run:

```
const _runId = Date.now().toString(36);
export const USER = {
  name: 'Haris Roy',
  email: `haris+${_runId}@mail.com`, // unique → reruns never hit "already
  registered"
  oldPassword: 'Password123',
  newPassword: 'Password456',
  // ...gender, dob, address, mobile
};
```

Why unique emails? The backend persists registrations. Suffixing the email with a base-36 timestamp means a second run never collides with leftover server state.

11. Lesson 4: The Full Account Lifecycle

The flagship test in `tests/advance/testApp.spec.ts` exercises the whole account lifecycle:



It is tagged `@regression @sanity` because it is the single most important confidence check: if this passes, the core app works.

A subtle but important pattern — **reading off-screen state from the tree, not from `getText()`**. `getText()` only sees *visible* nodes. To read a value that may be scrolled off-screen, walk the full accessibility tree:

```
import { collectAll } from './utils';

const nodes = collectAll(await screen.viewTree()); // full tree, visible or not
const value = nodes.find(n =>
  n.resourceId === id('tv_row_value') && n.isVisible)?.text;
```

12. Lesson 5: Negative Tests

Half the value of a suite is proving the app *rejects* bad input. The advanced suite has 12 negative tests — mismatched passwords, empty fields, duplicate email, wrong credentials, and so on. The pattern is always:

```
test('registration rejects mismatched confirm password',
  { tag: ['@negative'] }, async ({ screen, device }) => {
    const { freshLaunchToLogin, waitForErrorVisible, fillFreshText, id }
      = createHelpers(screen, device, APP_ID);

    await freshLaunchToLogin();
    // ...navigate to register, fill mismatched passwords, submit...

    const shown = await waitForErrorVisible(ERROR.passwordsMismatch); // soft poll
    expect(shown).toBe(true); // assert the
    boolean
  });
```

Expected error strings live in one place (`ERROR` in `utils.ts`), mirroring the app's `strings.xml` , so a wording change is a one-line fix:

```
export const ERROR = {
  passwordsMismatch: 'Passwords do not match',
  emailInvalid: 'Please enter a valid email address',
  invalidCredentials: 'Invalid email or password',
  // ...
};
```

Soft-assert, don't throw mid-poll. `waitForErrorVisible` returns `true` / `false` so you assert on the result. Throwing inside the polling loop would lose the diagnostic dump.

13. Lesson 6: Device Features and Gestures

`tests/advance/explore.spec.ts` goes beyond forms into gestures, runtime permissions, and system apps (contacts, camera, gallery, notifications).

Coordinate gestures use `device.driver` :

```
await device.driver.tap(x, y);
await device.driver.swipe('up', { distance: 650 });
await device.driver.swipe('right', { startX, startY, distance, duration });
await device.driver.longPress(x, y, 1200);
```

Make coordinates resolution-relative, not hardcoded pixels, so they work on the emulator *and* a higher-resolution phone:

```
import { getScreenSize } from './utils';
const { width, height } = getScreenSize(); // reads `adb shell wm size`
await device.driver.swipe('up', { startX: width / 2, startY: height * 0.8, distance:
600 });
```

Clean up system apps you launch. A camera/gallery left in the foreground will flake the *next* test.

`explore.spec.ts` force-stops those packages in `afterEach` :

```
test.afterEach(() => { deviceCleanup(); }); // BACK + HOME + force-stop
camera/gallery pkgs
```

Runtime permission dialogs are OS UI, not app UI. Handle both the *grant* and *deny* branches — the suite tests "allow → list shows" *and* "deny → list stays empty".

Part 4 — Write Your Own

14. Workshop: Write a New Test

Let's add a test that verifies the login screen rejects an **empty password**. We'll place it in the advanced suite and reuse the helpers.

Step 1 — Branch.

```
git checkout -b feature/login-empty-password
```

Step 2 — Add the test to `tests/advance/testApp.spec.ts` :

```
test('login is blocked with an empty password',
  { tag: ['@negative'] }, async ({ screen, device }) => {
    const { freshLaunchToLogin, fillFreshText, dismissKeyboard, waitForErrorVisible, id
  }
    = createHelpers(screen, device, APP_ID);

    // Arrange – clean slate, registered user
    await freshLaunchToLogin();
    // (register first so the email exists, then return to login)
    // ...reuse registerAndGoToLogin() if your scenario needs an account...

    // Act – type only the email, leave password empty, submit
    await fillFreshText(screen.getByTestId(id('et_login_email')), USER.email);
    await dismissKeyboard();
    await screen.getByTestId(id('btn_login')).tap();

    // Assert – the password-required error appears
    const shown = await waitForErrorVisible(ERROR.passwordRequired);
    expect(shown).toBe(true);
  });
```

Step 3 — Run just your test by title:

```
npx mobilewright test tests/advance/testApp.spec.ts \
  --grep "login is blocked with an empty password"
```

Step 4 — Make sure you didn't break smoke:

```
npx mobilewright test --grep @smoke
```

Step 5 — Open the report and confirm the step timings and (on failure) the screenshot:

```
npx mobilewright show-report
```

Stuck on a locator? You probably need to *discover* it first — that's the next section.

15. Discovering Locators with `inspect.spec.ts`

When you don't know an element's label or resource ID, dump the live accessibility tree:

```
npx mobilewright test tests/advance/inspect.spec.ts
```

It prints every visible node's `text`, `label`, `identifier`, and `resourceId`. Pick a locator by priority:

You see...	Use...
a <code>label</code>	<code>getByLabel('<label>')</code>
a <code>resourceId</code>	<code>getByTestId(id('<short_id>'))</code>
unique <code>text</code>	<code>getByText('<text>')</code>

You can also grab a screenshot of whatever is on screen right now:

```
npx mobilewright screenshot --output screen.png
```

16. Tagging and Selective Runs

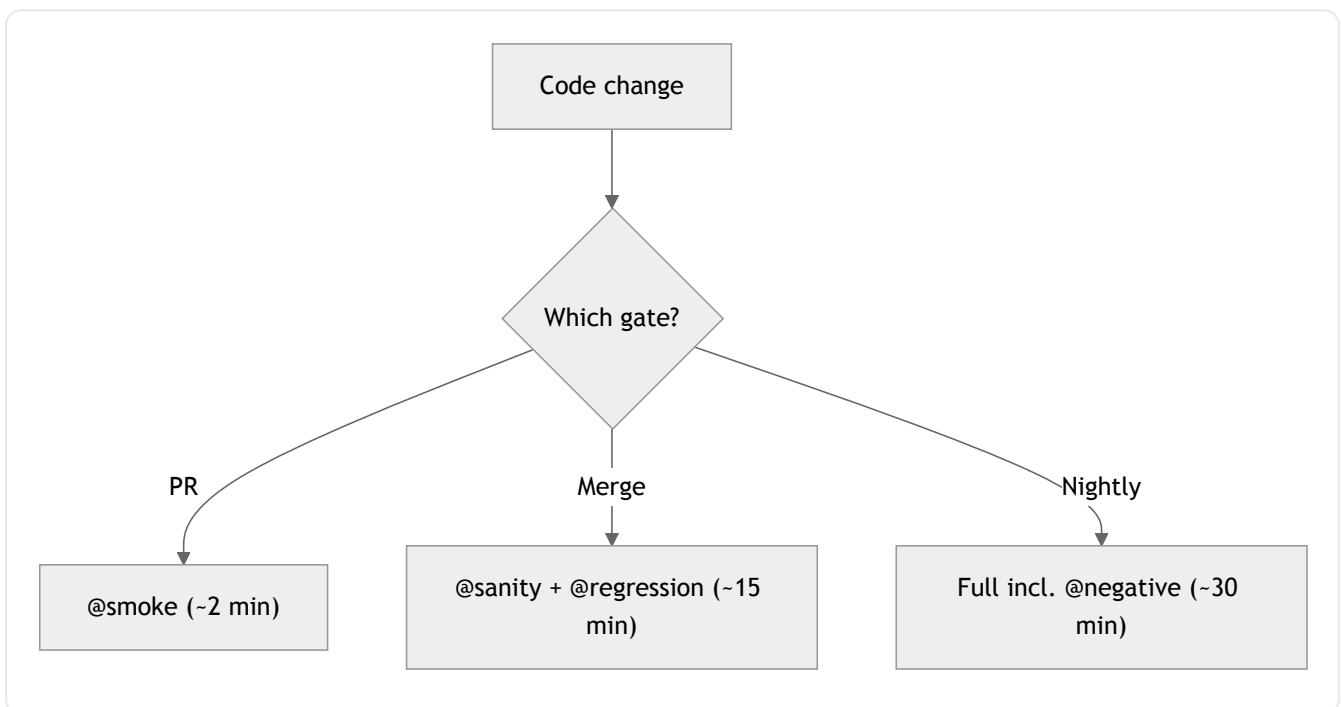
Every test carries tags so CI gates can pick subsets:

Tag	Meaning	Typical gate
@smoke	Fast confidence — app launches, core gestures work	PR / pre-commit
@sanity	Critical happy-path slices	Merge to main
@regression	Broad functional sweep	Merge to main
@negative	Validation / error-path coverage	Nightly

```

npx mobilewright test --grep @smoke           # one tag
npx mobilewright test --grep "@smoke|@sanity"  # alternation
npx mobilewright test --grep-invert @negative  # everything except negatives

```



Always tag a new test. An untagged test runs in `npm test` but is invisible to the targeted gates above.

Part 5 — Operate

17. Reading the Report

After any run, MobileWright writes an HTML report to `playwright-report/`.

```

npx mobilewright show-report           # open the last report
npx mobilewright test --reporter list  # streaming console output
npx mobilewright test --reporter json  # machine-readable, for dashboards

```

The report shows per-test status, console logs, step timings, and **screenshots on failure**. Per-test artifacts also land in `test-results/`. Both folders are git-ignored — they are build artifacts, not source.

To share: `zip -r report.zip playwright-report` or `npx serve playwright-report` on your LAN.

18. When Things Go Wrong

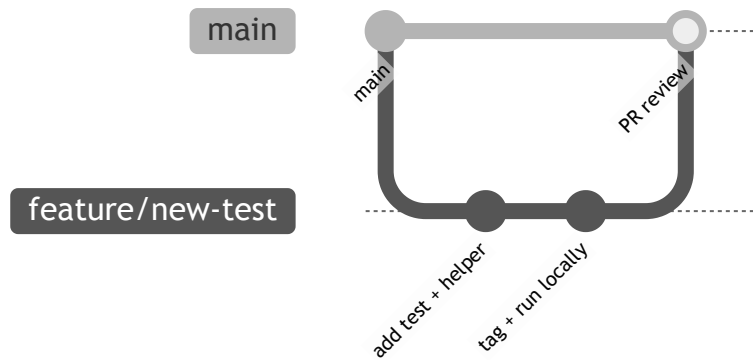
Symptom	Cause	Fix
<code>adb: more than one device/emulator</code>	Multiple devices attached	Specs auto-pick the physical device; override with <code>MW_DEVICE_ID</code> / <code>ANDROID_SERIAL</code>
<code>agent not installed</code>	Agent wiped (often after a reboot)	<code>npx mobilewright install</code>
<code>no XML content found in uiautomator dump</code>	UIAutomator crashed under load	<code>adb reboot</code> → wait 30 s → <code>npx mobilewright install</code> → rerun
Locator times out after scrolling	Element off-screen; <code>getText()</code> is visible-only	Read from <code>screen.viewTree()</code> / use <code>findVisibleElement([...], maxSwipes)</code>
Dropdown option not found	<code>AutoCompleteView</code> popups aren't in the a11y tree	Tap by computed coordinate (<code>selectDropDownItem</code>)
Typed text lands in the wrong field	Soft keyboard overlaps the next field	<code>await dismissKeyboard()</code> before filling the next field
Tests pass solo, flake in a batch	A previous test left a system app in foreground	Force-stop it in <code>afterEach</code> (see <code>explore.spec.ts</code>)

Quick connection reset and live logs:

```
adb kill-server && adb start-server      # reset the ADB daemon
adb logcat | grep com.haris.testapp      # tail the app's logs during a run
```

19. Checklist Before You Push

- ☐ Test starts from a clean slate (`freshLaunchToLogin()`).
- ☐ Uses the `USER` fixture and `id()` helper — no hard-coded data or resource IDs.
- ☐ Tagged appropriately (`@smoke` / `@sanity` / `@regression` / `@negative`).
- ☐ Cleans up any system app it launches.
- ☐ Passes locally: your test by title **and** `--grep @smoke` .
- ☐ Commit message is a short imperative subject (e.g. `add negative test for empty password`).



20. Where to Go Next

- **Reference everything** in [Part 6 — Reference](#) below — the full CLI/ADB command reference, locators, gestures, assertions, waits, design patterns, interview questions, and quick-reference tables, all folded into this guide.
- **Read the source.** The best documentation is `tests/advance/utiles.ts` ; every helper has a comment explaining the device quirk it works around.
- **Scale out.** With multiple physical devices, run files in parallel: `npx mobilewright test tests/advance/explore.spec.ts --workers 2` .
- **Wire up CI.** The [README](#) has a ready-to-use GitHub Actions workflow that boots an emulator, installs the agent, and gates PRs on `@smoke|@sanity` .
- **Contribute back.** Fork the repo at <https://github.com/sharisroy/mobilewright-testapp>, branch off `main` , and open a PR.
- **Consult the official docs** at <https://mobilewright.dev/> — the authoritative source for the framework's API, configuration, and platform support.

Part 6 — Reference

The MobileWright cheat sheet, folded into this tutorial. A dense, scannable reference for day-to-day work. Every example is drawn from this repository's working suite.

21. CLI Command Reference

```
npx mobilewright <command> [options]
```

Command	Purpose
<code>test [filter...]</code>	Run tests (optionally filtered by path)
<code>show-report [report]</code>	Open the last HTML report
<code>merge-reports [dir]</code>	Merge blob reports from sharded runs
<code>devices</code>	List connected devices/emulators/simulators
<code>screenshot</code>	Capture the current screen
<code>install</code>	Install the on-device agent
<code>doctor</code>	Validate the environment (Node, JDK, ADB, agent)
<code>init</code>	Scaffold a config + example test

test options

Option	Effect
<code>-c, --config <file></code>	Use a specific config
<code>--reporter <name></code>	<code>html list json</code>
<code>--grep <regex></code>	Run only matching titles/tags
<code>--grep-invert <regex></code>	Run everything except matches
<code>--retries <n></code>	Retry flaky tests
<code>--timeout <ms></code>	Override per-test timeout
<code>--workers <n></code>	Concurrency (capped at #devices)
<code>--shard <i/total></code>	Run one shard, e.g. 1/3
<code>--list</code>	List tests without running
<code>--pass-with-no-tests</code>	Exit 0 if nothing matches

```

npx mobilewright test                # all tests
npx mobilewright test tests/advance/testApp.spec.ts  # one file
npx mobilewright test --grep @smoke    # by tag
npx mobilewright test --grep "register, login"  # by title
npx mobilewright test --list           # dry list

```

22. ADB Command Reference

The single most important tool in mobile QA. This repo's helpers wrap many of these.

Connection

```
adb devices                # list devices
adb kill-server && adb start-server # reset the daemon
adb wait-for-device        # block until a device appears
adb -s <serial> <cmd>      # target one device
adb connect 192.168.1.50:5555 # wireless ADB
```

App lifecycle

```
adb install app.apk          # install
adb install -r app.apk      # reinstall, keep data
adb uninstall com.haris.testapp # remove
adb shell pm list packages | grep com.haris.testapp # is it installed?
adb shell pm clear com.haris.testapp # wipe app data (fresh state)
adb shell am force-stop com.haris.testapp # kill it
adb shell monkey -p com.haris.testapp -c android.intent.category.LAUNCHER 1 # launch
```

Permissions (used heavily in `explore.spec.ts`)

```
adb shell pm grant com.haris.testapp android.permission.CAMERA
adb shell pm revoke com.haris.testapp android.permission.READ_CONTACTS
adb shell pm grant com.haris.testapp android.permission.POST_NOTIFICATIONS
adb shell dumpsys package com.haris.testapp | grep 'POST_NOTIFICATIONS.*granted'
```

Input & keys

```
adb shell input keyevent 4 # BACK
adb shell input keyevent 3 # HOME
adb shell input keyevent 66 # ENTER
adb shell input keyevent 67 # DEL (backspace)
adb shell input keyevent 27 # CAMERA shutter
adb shell input keyevent 224 # WAKEUP
adb shell input keyevent 123 # MOVE_END (cursor to end of field)
adb shell input text "hello" # type text
adb shell input tap 540 1200 # tap a coordinate
adb shell input swipe 500 1500 500 500 200 # swipe (x1 y1 x2 y2 ms)
```

Diagnostics

```
adb logcat | grep com.haris.testapp # tail app logs
adb shell getprop sys.boot_completed # 1 == booted
adb shell dumpsys notification | grep test_app_channel # notification channel
adb shell cmd statusbar expand-notifications # open the shade
adb shell cmd statusbar collapse # close the shade
adb shell screencap -p /sdcard/s.png && adb pull /sdcard/s.png # screenshot
```

23. Emulator Management

```
emulator -list-avds                # list AVDs
emulator -avd Pixel_7_API_34 &    # boot one
emulator -avd Pixel_7_API_34 -no-snapshot -no-boot-anim -no-window &    # CI-friendly

adb wait-for-device                # wait for it
adb shell getprop sys.boot_completed    # 1 == fully booted

adb emu kill                       # shut it down
```

Create an AVD (CLI):

```
sdkmanager "system-images;android-34;google_apis;x86_64"
avdmanager create avd -n Pixel_7_API_34 -k "system-images;android-34;google_apis;x86_64" -d pixel_7
```

Tip: on CI, prefer `reactivecircus/android-emulator-runner` — it boots, caches, and tears down the emulator for you.

24. Locator Strategies (full)

Priority order: `getByLabel` → `getById` → `getByRole` → `getByText` → `getByType` .

```
// getByLabel – accessibility label (most stable) ✅
await expect(screen.getByLabel('Sign In Title')).toBeVisible();
await screen.getByLabel('Register Button').tap();

// getById – Android resource ID (very reliable) ✅
const id = (v: string) => `${APP_ID}:id/${v}`;
await screen.getById(id('btn_login')).tap();

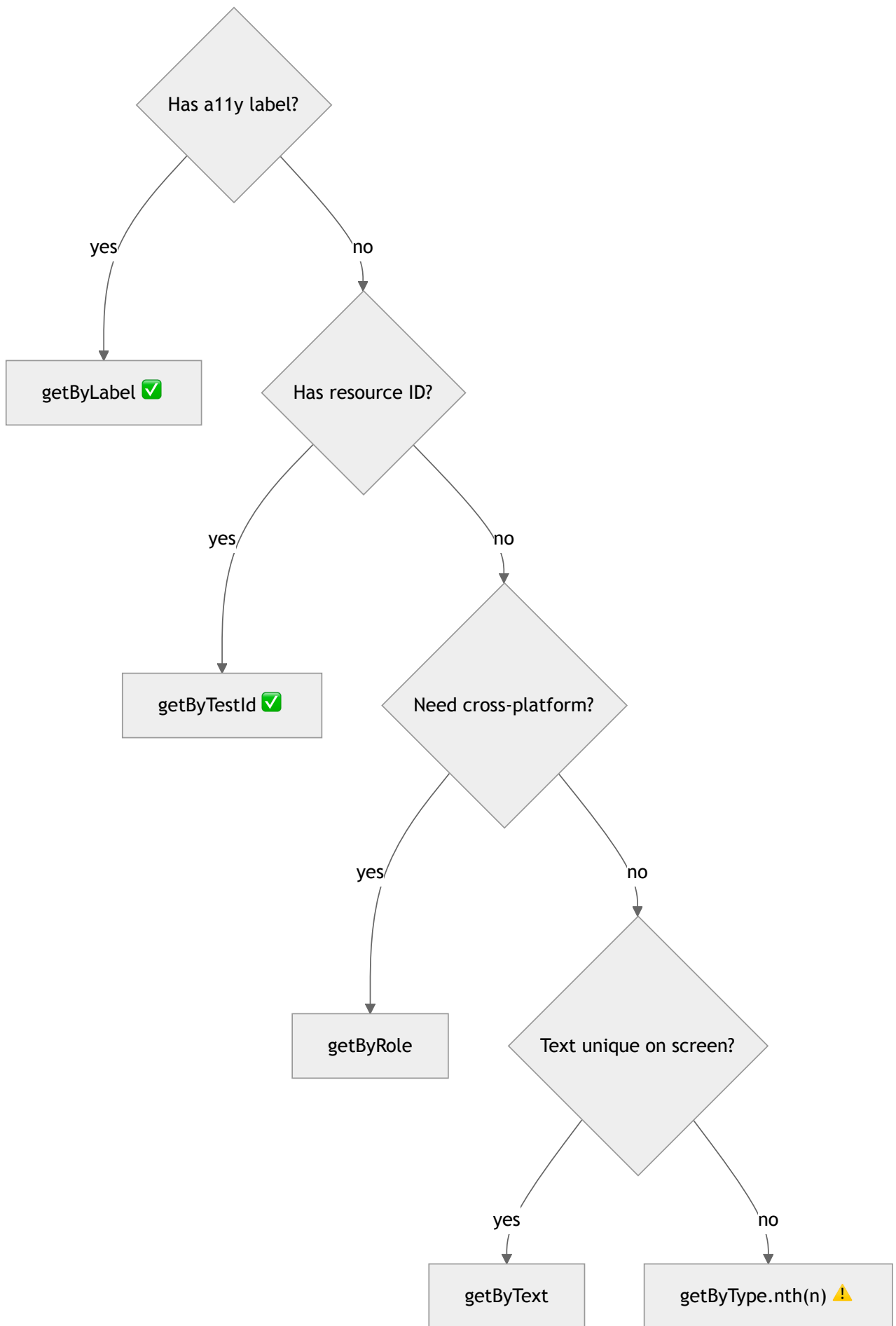
// getByRole – cross-platform semantic role
await screen.getByRole('button', { name: 'Login' }).tap();

// getByText – visible text (avoid for buttons/inputs)
await screen.getByText('My Profile').isVisible();
await screen.getByText(/manage your/i).getText();

// getByType – native component type (last resort) ⚠️
await screen.getByType('android.widget.EditText').nth(1).getText();

// getByPlaceholder
await screen.getByPlaceholder('Enter your email').fill('a@b.com');
```

Role	Android	iOS
button	Button , ImageButton	XCUICollectionElementKindButton
textfield	EditText	XCUICollectionElementKindText
text	TextView	XCUICollectionElementKindText
image	ImageView	XCUICollectionElementKindImage
switch	Switch	XCUICollectionElementKindSwitch
checkbox	CheckBox	—



25. Gestures (full)

```
// On a locator
await screen.getByLabel('Register Button').tap();
await screen.getByTestId(id('et_address')).fill('123 Main St');
await screen.pressButton('BACK');          // hardware/soft key

// On device.driver (coordinate-based)
await device.driver.tap(540, 1200);
await device.driver.swipe('up', { distance: 400 });
await device.driver.swipe('right', { startX: 90, startY: 980, distance: 900, duration: 150 });
await device.driver.longPress(centre.x, centre.y, 2500);
```

Scroll until visible (the `findVisibleElement` pattern)

```
const mobileField = await findVisibleElement([
  { name: 'et_mobile (testId)', locator: screen.getByTestId(id('et_mobile')) },
  { name: 'Mobile (label)', locator: screen.getByLabel('Mobile Number Edit Text')}
], 5);
await mobileField.fill('+912345678');          // swipe up to 5 times
```

Tapping inside a scroll container

When a nested list eats your swipe, anchor to the page edge:

```
// Drive the OUTER page scroll, not the inner Contacts list
device.driver.swipe('up', { startX: 1040, startY: 1700, distance: 600, duration: 250 });
```

26. Assertions & Wait Strategies

Web-Playwright-style, auto-retrying until the global timeout.

```
await expect(locator).toBeVisible();
await expect(locator).not.toBeVisible();
await expect(locator).toBeEnabled();
await expect(locator).toHaveText('My Profile');    // exact
await expect(locator).toContainText('Profile');    // substring
await expect(locator).toHaveText(/profile/i);      // regex
await expect(locator).toBeVisible({ timeout: 15_000 });

// Value assertions on extracted data
const name = await screen.getByLabel('Profile Name').getText();
expect(name).toBe('Haris Roy');
```

Auto-retrying (`expect(locator).toBeVisible()`) vs one-shot (`expect(value).toBe(...)`):
the first polls the live UI; the second checks a value you already pulled. Use the first for UI state, the second for extracted data.

Prefer auto-retrying assertions over fixed sleeps

```
// ✅ good – waits exactly as long as needed, up to the timeout
await expect(screen.getByLabel('Sign In Title')).toBeVisible({ timeout: 20_000 });

// ⚠️ avoid – wastes time or is too short
await sleep(3000);
```

Poll a condition (this repo's `waitForTextVisible` / `waitForErrorVisible`)

```
const waitForTextVisible = async (text: string, timeout = 20_000): Promise<boolean> =>
{
  const deadline = Date.now() + timeout;
  while (Date.now() < deadline) {
    if (await screen.getByText(text).isVisible().catch(() => false)) return true;
    await sleep(500);
  }
  return false;
};
```

27. Reading the View Tree

`getText()` only works on **visible** elements. To read off-screen values or discover locators, dump the whole accessibility tree.

```
const collectAll = (nodes: any[], out: any[] = []): any[] => {
  for (const n of nodes) { out.push(n); if (n.children?.length) collectAll(n.children, out); }
  return out;
};

const nodes = collectAll(await screen.viewTree());
const node = nodes.find(n => n.resourceId === id('et_full_name') || n.identifier === id('et_full_name'));
const value = node?.text ?? '';
```

Each node exposes: `type`, `label`, `text`, `resourceId` / `identifier`, `isVisible`, `bounds` (`x/y/width/height` or `left/top/right/bottom`), and `children`.

Tree field	Locator to use
label="..."	getByLabel('...')
resourceId="..."	getByTestId('...')
text="..."	getByText('...')
type="..."	getByType('...')

Tapping elements not in the a11y tree

Android `AutoCompleteView` popups don't appear in the tree. Compute a coordinate below the field and tap it (this repo's `selectDropdownItem`):

```
const field = nodes.find(n => n.resourceId === id('acv_gender'));
const { x, y, width, height } = field.bounds;
await device.driver.tap(Math.round(x + width / 2), Math.round(y + height + 50));
```

28. Runtime Permissions

Drive permissions deterministically with ADB instead of fighting OS dialog text (which varies by Android version/OEM). Straight from `explore.spec.ts`:

```
const { adbShell } = createHelpers(screen, device, APP_ID);

adbShell(`pm revoke ${APP_ID} android.permission.READ_CONTACTS`); // known starting
state
adbShell(`pm grant  ${APP_ID} android.permission.CAMERA`);        // skip the dialog
entirely
adbShell('input keyevent 4');                                       // dismiss a dialog
with BACK
```

Deliver and read a push notification:

```
execSync(`adb shell "cmd notification post -S bigtext -t 'Title' tag 'Body text'"`);
execSync('adb shell cmd statusbar expand-notifications');
const shade = collectAll(await screen.viewTree());
expect(shade.find(n => n.text === 'Title')).toBeTruthy();
execSync('adb shell cmd statusbar collapse');
```

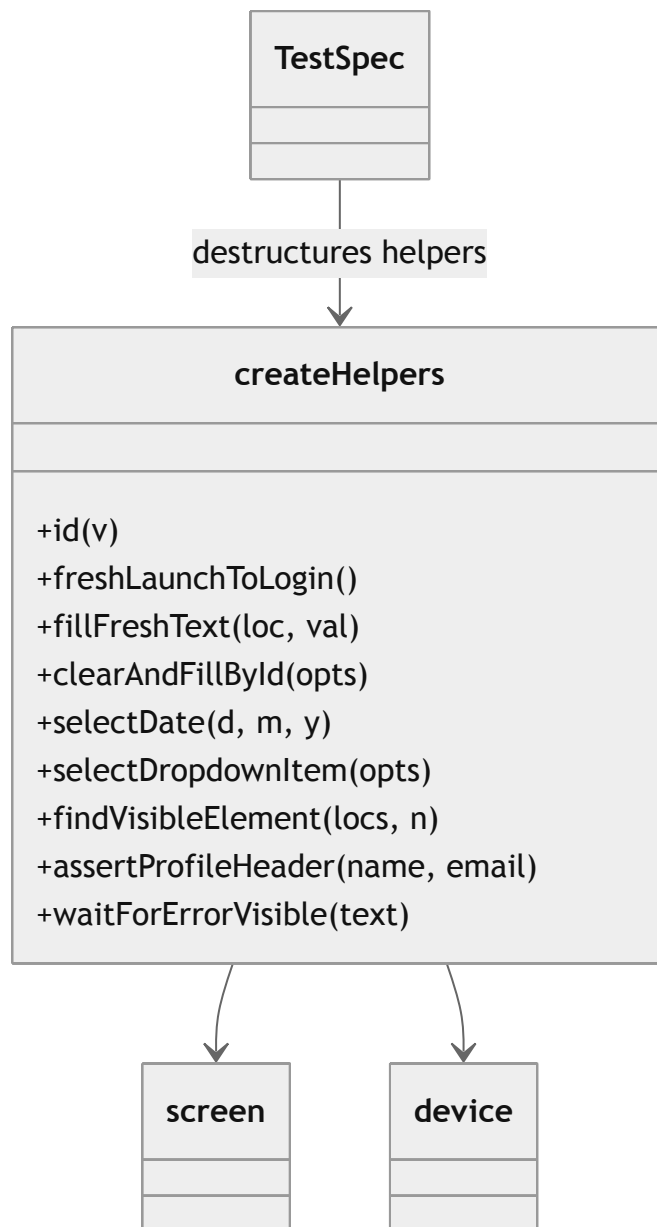
29. API Testing

MobileWright focuses on UI, but you can mix in HTTP checks with any Node client — `fetch` (Node 18+) or `axios` — inside the same spec to validate that the UI and backend agree.

```
test('profile API reflects the UI update', async ({ screen, device }) => {  
  // 1. drive the UI to update the profile (omitted)  
  
  // 2. assert the backend agrees  
  const res = await fetch('https://api.example.com/users/me', {  
    headers: { Authorization: `Bearer ${token}` },  
  });  
  expect(res.status).toBe(200);  
  expect((await res.json()).email).toBe(USER.email);  
});
```

Pattern: seed/clean data over the API for speed, then assert the *outcome* through the UI (or vice-versa). Keep request helpers in a shared module, like `utils.ts`.

30. Framework Design Patterns



- **Helper factory** (used here) — `createHelpers(screen, device, APP_ID)` returns a closure of functions bound to one test fixture. Lighter than classes; easy to share across files.
- **Page Object Model** — for larger suites, wrap each screen in a class exposing locators + actions. Either works; this repo chose the factory for brevity.
- **Self-healing locators** — `findVisibleElement([...candidates], maxSwipes)` tries each candidate, scrolls, and retries.
- **Deterministic data** — a once-evaluated `runId` gives every test in a run the same unique email, so reruns never collide.
- **Clean-slate setup** — `freshLaunchToLogin()` does `pm clear` + relaunch so no test depends on another.
- **System-app cleanup** — force-stop camera/gallery packages in `afterEach` so the next test can foreground the app.

31. Best Practices

1. Respect the locator priority: **label** → **testId** → **role** → **text** → **type**.
 2. Keep a DRY `id()` helper; never hard-code `com.haris.testapp:id/...` inline.
 3. Read off-screen state from `viewTree()`, not `getText()`.
 4. Start every test clean (`freshLaunchToLogin()`); don't chain test dependencies.
 5. Use unique, run-scoped data (`haris+<runId>@mail.com`).
 6. Prefer auto-retrying `expect(...).toBeVisible()` over `sleep()`.
 7. Keep `timeout` (per-action) modest; raise the **test** ceiling with `test.setTimeout()`.
 8. Tag every test (`@smoke` / `@sanity` / `@regression` / `@negative`).
 9. Soft-assert error messages by polling the tree and asserting a boolean.
 10. Clean up any system app you launch, in `afterEach`.
 11. Centralize selectors, data, and error strings in a shared module.
 12. In CI, use `npm ci`, boot the emulator before installing the agent, and always upload the report.
-

32. Interview Questions

Beginner

1. What problem does MobileWright solve, and how does it differ from Appium? (*No external server; bundles an on-device agent; Playwright-style API.*)
2. What's the difference between the `screen` and `device` fixtures? (`screen` = *locators/queries*; `device` = *app lifecycle + raw driver gestures*.)
3. Name the locator strategies in priority order. (`getByLabel` → `getByTestId` → `getByRole` → `getText` → `getByType`.)
4. How do you install the on-device agent, and when must you reinstall it? (`npx mobilewright install`; *after a reboot or device wipe.*)

Intermediate

5. Difference between `expect(locator).toBeVisible()` and `expect(value).toBe(...)`? (*First auto-retries against live UI; second is a one-shot check on extracted data.*)
6. How do you read a value from a field that's scrolled off-screen? (`collectAll(await screen.viewTree())` then find the node — `getText()` is visible-only.)
7. How are slow end-to-end tests kept from timing out without inflating per-action timeouts? (`test.setTimeout()` raises the test ceiling independently of the global `timeout`.)
8. How do you target one of several connected devices? (`ANDROID_SERIAL=...` / `MW_DEVICE_ID=...` `env var` or `--device <serial>`.)

Advanced

9. How do you interact with an element that isn't in the accessibility tree (e.g. an `AutoCompleteView` dropdown)? (*Compute its bounds from the tree and `device.driver.tap(x, y)` below the field.*)

10. Why does this suite use a unique email per run, and how is it generated? (Avoid "already registered" on reruns; `Date.now().toString(36)` suffix evaluated once.)
11. Why `workers: 1` with `fullyParallel: true` in the advance config? (Tests share one account/state → sequential within a worker, but files can still parallelize across devices.)
12. A test passes alone but flakes in the batch — how do you diagnose and fix it? (A prior test left a system app foregrounded; force-stop it in `afterEach`; inspect with a `viewTree()` dump.)

33. Quick-Reference Tables

CLI

Task	Command
Run all tests	<code>npx mobilewright test</code>
Run a file	<code>npx mobilewright test tests/advance/testApp.spec.ts</code>
Run by tag	<code>npx mobilewright test --grep @smoke</code>
Run by title	<code>npx mobilewright test --grep "register, login"</code>
List tests	<code>npx mobilewright test --list</code>
Open report	<code>npx mobilewright show-report</code>
List devices	<code>npx mobilewright devices</code>
Install agent	<code>npx mobilewright install</code>
Screenshot	<code>npx mobilewright screenshot --output s.png</code>
Env doctor	<code>npx mobilewright doctor</code>
Merge shards	<code>npx mobilewright merge-reports ./blob-reports</code>

ADB & key codes

Task	Command	Key	Code
List devices	<code>adb devices -l</code>	BACK	4
Install APK	<code>adb install -r app.apk</code>	HOME	3
Wipe app data	<code>adb shell pm clear com.haris.testapp</code>	ENTER	66
Force-stop	<code>adb shell am force-stop com.haris.testapp</code>	DEL	67
Grant permission	<code>adb shell pm grant <pkg> <perm></code>	CAMERA	27
Revoke permission	<code>adb shell pm revoke <pkg> <perm></code>	WAKEUP	224
Tail logs	<code>adb logcat grep com.haris.testapp</code>	MOVE_END	123

Locators, assertions & gestures

Locator	Use when	Assertion	Meaning
<code>getByLabel(l)</code>	Has an a11y label (preferred)	<code>toBeVisible()</code>	Element is shown
<code>getById(id)</code>	Has a resource ID	<code>not.toBeVisible()</code>	Element is hidden/absent
<code>getByRole(r, {name})</code>	Cross-platform role	<code>toBeEnabled()</code>	Element is interactable
<code>getByText(t)</code>	Text is unique on screen	<code>toHaveText(t)</code>	Exact text match
<code>getByType(t)</code>	Last resort, by type	<code>toContainText(t)</code>	Substring match
<code>getByPlaceholder(p)</code>	Input with placeholder	<code>toHaveText(/re/i)</code>	Regex match

Gesture	Call
Tap (locator)	<code>locator.tap()</code>
Fill	<code>locator.fill('text')</code>
Tap (coord)	<code>device.driver.tap(x, y)</code>
Swipe	<code>device.driver.swipe('up', { distance })</code>
Swipe (precise)	<code>device.driver.swipe('right', { startX, startY, distance, duration })</code>
Long press	<code>device.driver.longPress(x, y, ms)</code>
Hardware key	<code>screen.pressButton('BACK')</code>

That's the whole framework, end to end. Happy testing — and when in doubt, the [official docs](#) are the source of truth.

Attribution, repository, and the full legal notice are on the next page — see [About This Document](#).

About This Document

Title	MobileWright Tutorial & Reference
Author	Haris Chandra Roy
App under test	Mobile QA Sandbox (com.haris.testapp)
Repository	https://github.com/sharisroy/mobilewright-testapp
Official docs	https://mobilewright.dev/
Generated	June 11, 2026

Disclaimer & legal notice. This is a personal, unofficial learning project created and maintained by Haris Chandra Roy. It is **not** an official MobileWright product and carries **no legal binding, warranty, or guarantee** of any kind — it is provided "as is" for educational purposes only, and you use it at your own risk. "MobileWright" and any related names, logos, and trademarks are the property of their respective owners; this document is not affiliated with, endorsed by, or sponsored by them. For authoritative, up-to-date information always defer to the official documentation at <https://mobilewright.dev/> — where this guide and the official docs disagree, the official docs win. Nothing here is professional, legal, or contractual advice.

— End of document —